



Kauno technologijos universitetas

Informatikos fakultetas

Tikslu orientuoto veiksmų planavimo simuliacija

T120B199 Dirbtinis intelektas žaidimams

Aidas Litvinas

Projekto autorius

Doc. dr. Evelina Stanevičienė

Dėstytoja

Kaunas, 2025

Turinys

Įvadas.....	3
1. Panašių sprendimų analizė.....	3
2. Projekto koncepcija ir planas.....	4
3. Projektavimas.....	5
4. Realizavimas.....	7
5. Analizė ir refleksija.....	14
Išvados.....	14
Literatūra ir šaltiniai.....	14

Ivadas

Šio projekto tikslas yra sukurti žaidimo veikėjų DI simuliaciją, kurioje agentai valdomi tikslu orientuoto veiksmų planavimo (angliškai - goal oriented action planning, sutrumpinus - GOAP) modeliu. Žaidimo aplinkoje turi veikti bent keli agentai, kurie pademonstruoja šio DI modelio pranašumą: galimybę priskirti dinamiškus veiksmų planus, kurie gali būti skirtingi priklausant nuo žaidimo aplinkos bei agento būseną.

1. Panašių sprendimų analizė

GOAP modelis buvo sukurtas pagal "Stanfordo tyrimų instituto problemų sprendėjo" modelį (angl. akronimas - STRIPS) ir pirmą kartą buvo panaudotas "F.E.A.R." žaidime [1].

GOAP modelio statymo blokai yra:

1. Pasaulio būsena (angl. vadinama "blackboard") - tai paprastai yra Dictionary, kuris saugo žaidimo pasaulio informaciją, dažniausiai Boolean reikšmėmis. Kiekvienas agentas gali turėti savo "blackboard" kopiją, kurioje pasilieka tik sau aktualias reikšmes.

Pavizdys kaip gali atrodyti agento "blackboard":

```
{
    "is_hurt": true,
    "has_weapon": false,
    "has_potion": true,
    "visible_enemy": true,
    "inventory_full": false
}
```

Kiekvienos būsenos pokytis verčia agentui persvarstyti, koks jo tikslas.

2. Tikslas. Tai yra kokia nors įvardinta pasaulio būsenos sąlyga bei norimas rezultatas, pavyzdžiui:

Tikslas: recover_health

Sąlygos: { "is_hurt": true }

Norimas rezultatas: { "is_hurt": false }

Tikslas: collect_materials

Sąlygos: { "is_hurt": false, "inventory_full": false }

Norimas rezultatas: { "inventory_full": true }

Kiekvienas tikslas turi atitikti konkrečias pasaulio būsenas, todėl bet kuriai iš jų pasikeitus reikia perskaičiuoti, ar nepasikeitė tikslas. Tikslui pasikeitus, senas planas išmetamas lauk ir suplanuojama nauja veiksmų seka.

3. Veiksmas. Tai yra plano statymo blokas - kiekvienas veiksmas turi kokias nors pasaulio būsenos sąlygas, bei efektus. Kartu veiksmai gali turėti nustatytą kainą, kuri leidžia planuotojui parinkti optimalų veiksmų planą. Veiksmo pavyzdys:

Veiksmas: `drink_potion`

Sąlygos: `{ "has_potion": true }`

Efektai: `{ "is_hurt": false, has_potion: false }`

4. Planas. Planuotojas pagal aktyvų tikslą parenka veiksmų seką, kurią tada agentas pradeda vykdyti. Planuotojas sugeneruoja kelis skirtingus planus, galiausiai parenka vieną iš jų pagal tai, kurio plano veiksmų kainų suma mažesnė.

Artimiausias alternatyvus modelis yra "Hierarchical task network planning" (HTN). Šiame modelyje yra dviejų tipų veiksmai - primitivūs bei abstraktūs. Abstraktus veiksmas tai tiesiog kelių primitivių arba abstrakčių veiksmų junginys. Šis modelis naudojamas "Killzone 3" ir "Transformers 3: Fall of Cybertron" žaidimuose [2], bei daugelyje kitų žaidimų.

Bendrai paėmus, abu šie metodai yra kažkokia automatizuoto planavimo forma. Jie naudoja gana brangius apskaičiavimus, todėl paprastai naudojami tik aukšto biudžeto "AAA" žaidimuose, kurie reikalauja sudėtingų DI agentų.

2. Projekto koncepcija ir planas

Šiam projektui naudosiu jau sukurtą projektą, kurį kūriau 2024 m. rudens semestro "Žaidimų kūrimo pagrindai" modulio metu [3]. Tai yra šešiakampių tinklo "turn-based" strateginis žaidimas, kuriame du žaidėjai gali išdėstyti ir kontroliuoti žmogeliukus kovos lauke. Šiam projektui kovojimo dalies tikriausiai nenaudosiu, noriu tik pasiimti šešiakampio tinklo pasaulio pagrindą, kuriame galėčiau paleisti kelis žmogeliukų agentus ir stebėti jų veikimo simuliaciją. Šie žmogeliukai yra kelių skirtingų tipų, mechanškai skiriasi tik skaičiais atributuose.

Žaidimo pasaulyje noriu turėti šios objektus, su kuriais gali sąveikauti agentai:

1. Laužas. Agentų tikslas yra nešioti medieną iki laužo, kad neužgestų ugnis, kuri per laiką silpnės.
2. Medžiai. Agentas priėjęs prie medžio gali jį nukirsti, taip į rankas pasiimdamas medienos krūvelę.
3. Spygliuoti krūmai. Tai tinklo langelis, pro kurį praeinant agentas bus kažkiek sužalotas.
4. Medicinos laukelis. Tai tinklo langelis, kuris, priešingai nuo krūmų, pagydo agentą.

Agentų navigacija naudoja Godot pateiktą A* navigacijos algoritmą, bet galimai teks parašyti truputėlį modifikuotą algoritmą, kuris atsižvelgtų į krūmų perėjimo kainą. Kartu GOAP implementacijai pasitelksiu į pagalbą "GdPlanningAI" įskiepi [4].

3. Projektavimas

Pasaulio būseną išsaugo šią informaciją:

- Laužo ugnies stiprumas.
- Medienos kiekis pasaulyje.
- Medžių kiekis pasaulyje.

Kiekvienas agentas turi savo atskirą būseną, kurioje saugos:

- Savo poziciją.
- Laikomą objektą.
- Gyvybių taškus.

Tikslai, kuriuos agentas gali turėti, yra:

- Tikslas: wander_goal
Sąlygos:
Norimas rezultatas: Agento pozicija pasikeičia.
- Tikslas: maintain_bonfire_goal
Sąlygos: Laužo stiprumas žemesnis nei 80%.
Norimas rezultatas: Laužo stiprumas lygus 80% arba daugiau.
- Tikslas: heal_goal
Sąlygos: Agento gyvybė žemesnė nei laužo stiprumas.
Norimas rezultatas: Agento gyvybė aukštesnė nei laužo stiprumas.

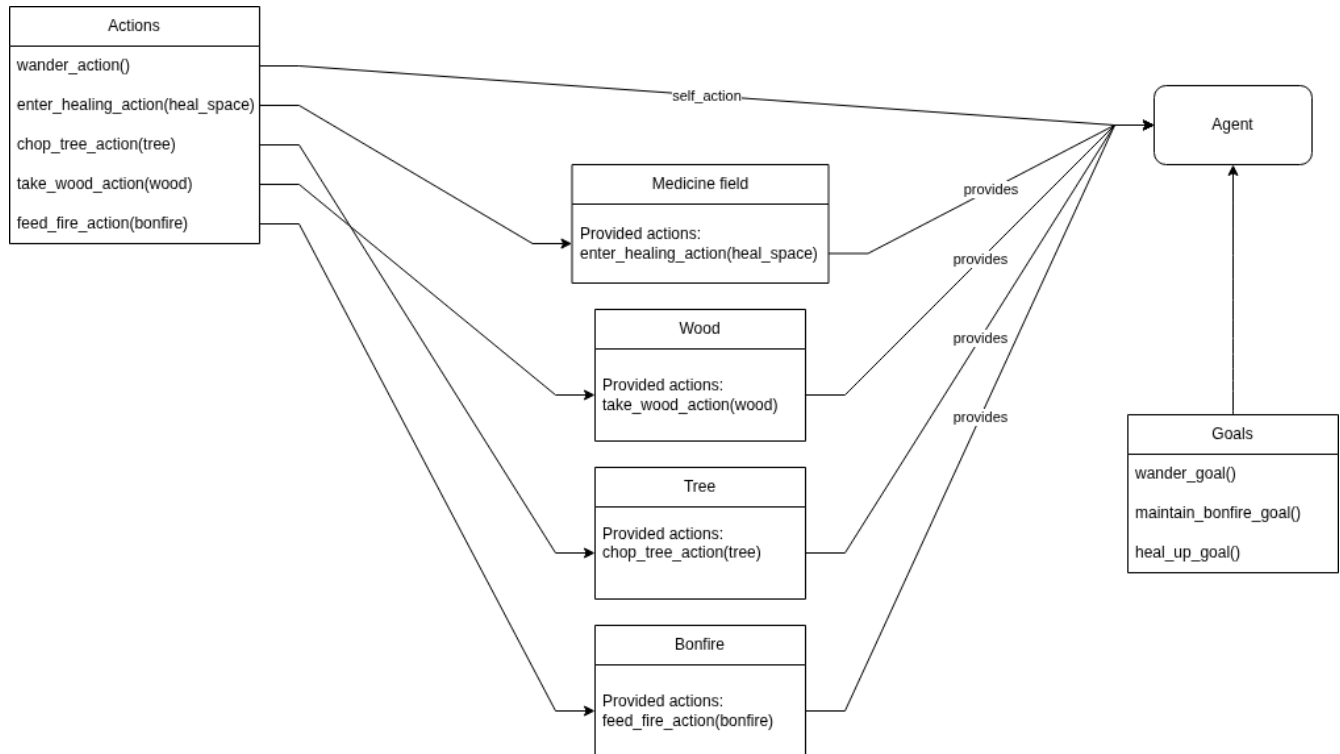
Galimi veiksmai:

- Veiksmas: wander_action
Sąlygos:
Efektai: Pozicija pasikeičia, agentas pajuda viena plytele į bet kurią pusę
- Veiksmas: enter_healing_action
Sąlygos: Tikslas pozicija yra pasiekiamas.
Efektai: Agento pozicija yra lygi gydymo lauko pozicijai, agento gyvybės taškai papildomi.
- Veiksmas: chop_tree_action
Sąlygos: Tikslas pozicija yra pasiekiamas. Agentas stovi šalia medžio.
Efektai: Medis dingsta, išmetama mediena medžio senoje pozicijoje.

- Veiksmas: `take_wood_action`
Sąlygos: Tikslas pozicija yra pasiekama. Agentas stovi šalia medienos.
Efektai: Agentas pradeda laikyti medienos objektą.
- Veiksmas: `feed_fire_action`
Sąlygos: Tikslas pozicija yra pasiekama. Agentas laiko medieną. Agentas stovi šalia laužo.
Efektai: Mediena dingsta. Laužo stiprumas pakyla 20%. Agentas praranda 15 gyvybių taškų (nudega rankas)

Aprašius šiuos komponentus, agentai turėtų turėti visą informaciją, kad galėtų atlikt simuliaciją.

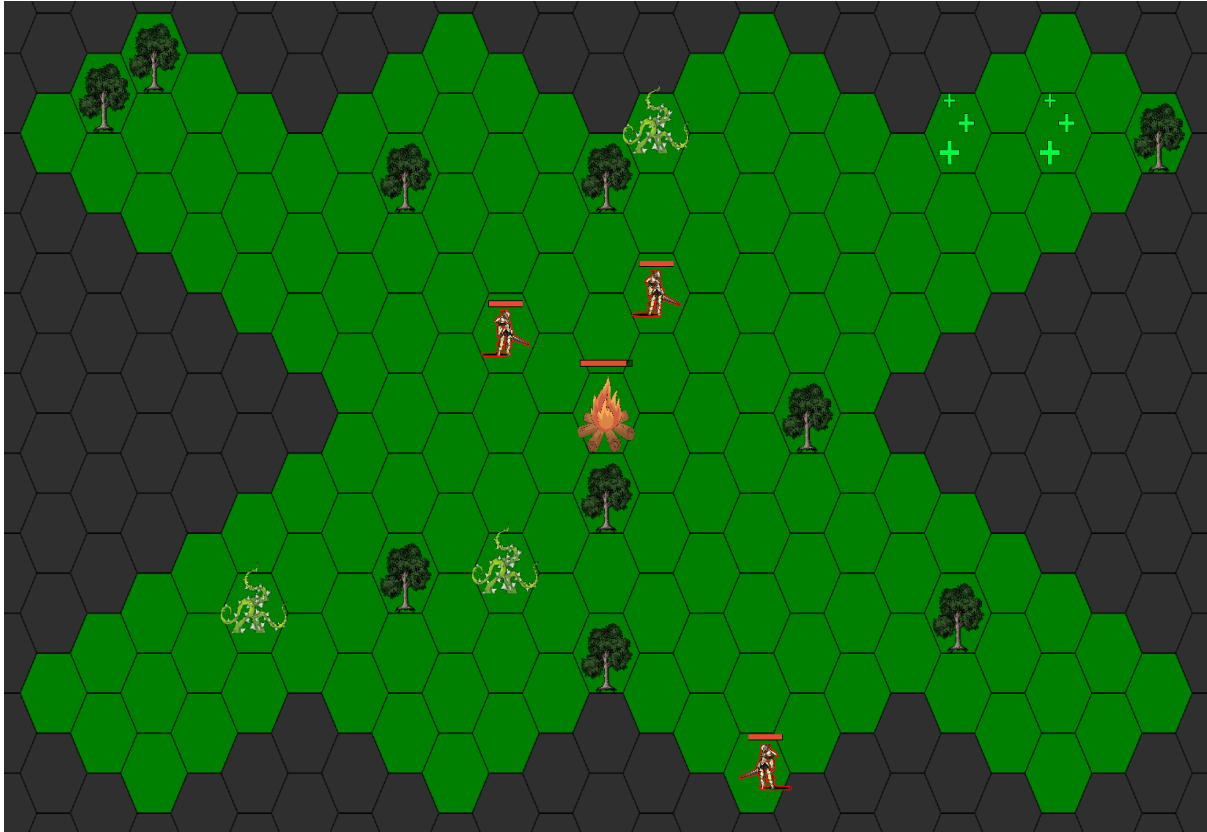
Komponentų diagrama:



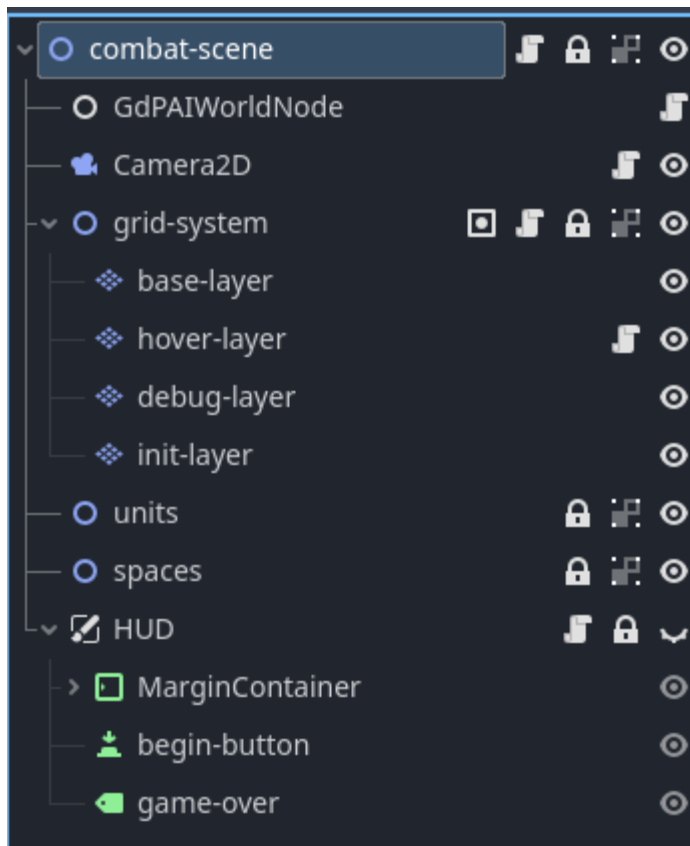
4. Realizavimas

Projektui naudojamas Godot 4.5 variklis, bei GdPlanningAI įskiepis. Jis suteikia GOAP tikslų suteikimo ir planų formavimo funkcionalumą, bei tam tikras abstrakcijas, kurios leidžia aprašinėti komponentus panašia forma, kaip ir planavimo etape. Teko pačiam pritaikyti šią sistemą, kad veiktų su mano ankstesnio projekto plytelių tinklo sistema, bei ėjimų pagrindo sistema.

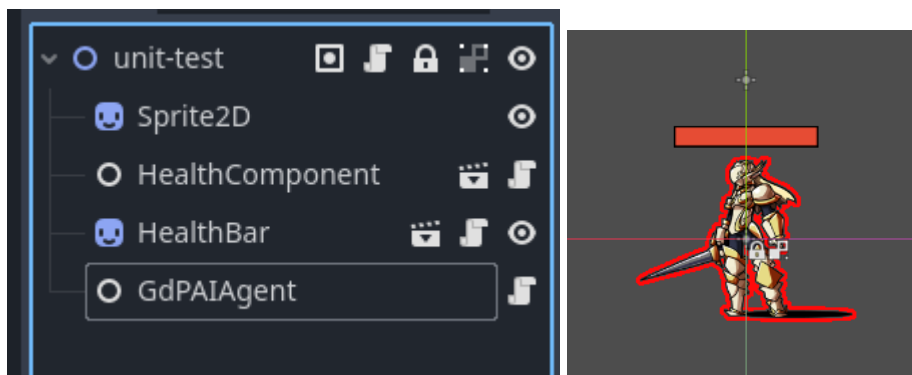
Žaidimo pasaulio vaizdas:



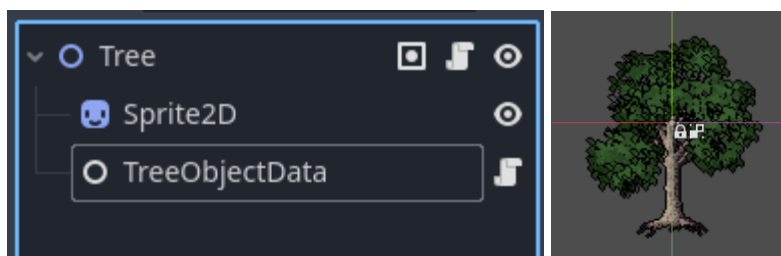
Pasaulio scenos struktūra:



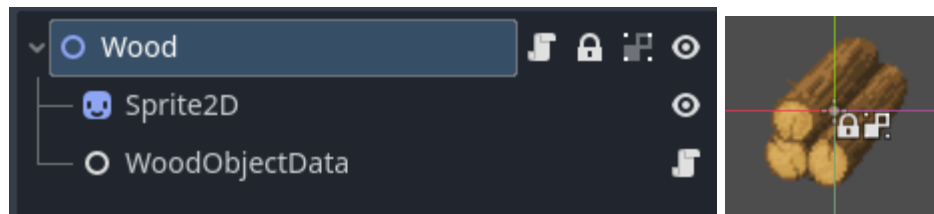
Žmogeliuko/agento scenos struktūra:



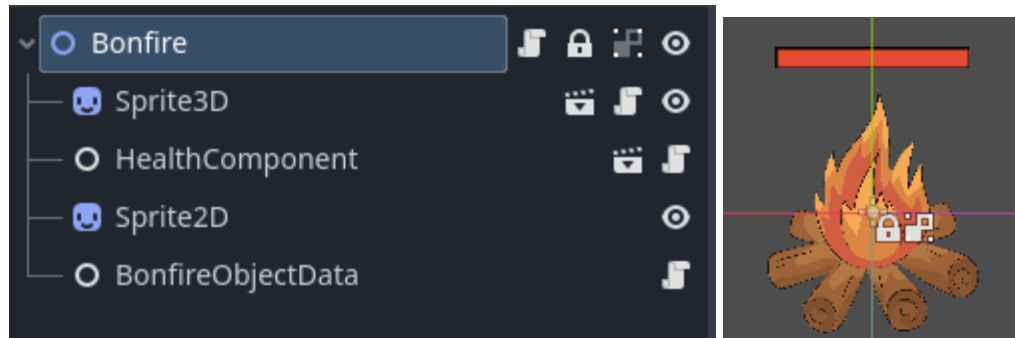
Medžio scenos struktūra:



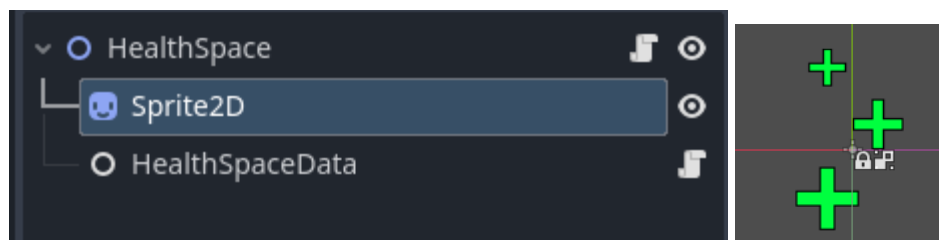
Medienos scenos struktūra:



Laužo scenos struktūra:



Medicinos lauko scenos struktūra:



Spygliuoto krūmo scenos struktūra:



Mano plytelių lauko sistema naudoja Hex formos langelių tinklą, kuriame naudoju AStar2D objektą. Šiam objektui paleidžiu savo parašytą BFS algoritmą, kuris sukuria ir sujungia langelių mazgus. Toliau pateikiu kitiems žaidimo objektams galimybę gauti navigacijos kelią, bei visokius kitus pagalbinius metodus. Base-layer tinklo sluoksnis nusako pasaulio aplinką. Čia vyksta navigacija ir mazgų valdymas navigacijos manipuliacijai. Hover-layer sluoksnis yra pelės žymekliui, vizualiai pavaizduoja kur dedamas žmogeliukas.

Mano ėjimų sistema yra šakniam scenos mazge, kode vadinama kaip „play_loop“. Ji žaidimo pradžioje laukia, kol HUD išdėstys pagal žaidėjo paspaudimus žmogeliukus, ir tada paleidžia ėjimų veiksmą. Žmogeliukai yra eilėje, savo ėjimą įvykdę jie numetami į eilės galą. Kiekvieną ėjimą agentui suteikiamas leidimas vykdyti ėjimą, agentas gavęs leidimą gauna pagal tikslą suformuotą planą, ir pradėjęs jį vykdyti, po kiekvieno veiksmo žingsnio vykdymo leidžia sistemai perduoti ėjimą kitiems agentui.

Sistemos DI sąveikaujantys elementai yra:

- Pasaulio GdPAIWorldNode mazgas. Jis laiko pasaulio būseną tam skirtam GdPAIBlackboard resurse.
- Agento GdPAIAgent mazgas. Jis sąveikauja su agento kodu. Pats atlieka plano kūrimo logiką, bei leidžia agento kodui sąveikauti su agento individualiu blackboard resursu.
- Pasaulio objektų (medžių, medienos, laužo) GdPAIObjectData DI duomenų mazgas. Kiekvienas pasaulio objektas, kuris sąveikauja su agentais, bet pats nėra agentas, turi šiam objekte aprašyti kokius įmanomus veiksmus suteikia agentams, bei duoda metodą leidžiantį planavimo etape agentams paimti objekto kopiją simuliacijai.
- Tikslai. Kiekvienas tikslas yra individualus script failas.
- Veiksmai. Kiekvienas veiksmas yra individualus script failas.
- Išankstinės sąlygos. Šie Precondition objektai įvairiose vietose naudojami aprašyti logiką, kuri priklauso nuo blackboard būsenų.

Tikslai

Tikslas turi šiuos metodus:

- `compute_reward`. Čia apskaičiuojama kaina, pagal kurią apskaičiuojamas, tikslo radimas.
- `get_desired_state`. Šis metodas turi grąžinti Precondition sąrašą, pagal kurį nustatoma, kokios pasaulio būsenos nusako, ar tikslas įvykdytas.

Visi tikslai suteikiami agentui į jo tikslų sąrašą.

Veiksmai

Viksmai turi šiuos metodus, kurie eilės tvarka vykdomi plano radimo ir vykdymo procese.

- `get_validity_checks`. Tai Precondition patikrinimų sąrašas, kurie patikrinami kiekvieną kartą, kai yra parenkamas naujas tikslas.
- `get_action_cost`. Tai veiksmo kaina, planas suranda patį pigiausią kelia, jeigu čia grąžinama INF vertė, veiksmas nebėra svarstomas plano kūrime.
- `get_preconditions`. Tai visokie patikrinimai, kurie plano formavimo metu tikrina, ar juos vykdyti galima. Pavyzdžiui, jeigu veiksmas yra pajudėti, tai čia būtų vykdomas patikrinimas, ar agento dabartinė pozicija yra vienoda, kaip ir tikslo. Jei vienoda, veiksmas nebėra svarstomas.
- `simulate_effect`. Šis metodas susikuria agento ir pasaulio blackboard kopijas, ir susimuliuoja jų efektus. Tai reikia daryti norint žinoti veiksmo pradinės ir galinės būsenos pokyčius. Juos naudojant plano formavimas vykdo kelio radimą, naudodamas kiekvieną veiksmą kaip mazgus, ir šiuos mazgus ir jungia šie būsenos pokyčiai.
- `reverse_simulate_effect`. Tai panašu kaip ir praeitis metodas, tik simuliacija atliekama atvirkštine tvarka. To gali prireikti plano formavimo kelio radimo proceso optimizavimui.
- `pre_perform_action`. Kodas, kuris atlieka visus veiksmus, kad paruoštų agentą ir būsenas veiksmo vykdymui.
- `perform_action`. Vykdoma veiksmo logika.
- `post_perform_action`. Kodas, kuris atstato būsenas atgal į normalią būseną.

Pavyzdžiui, mano Klajojimo tikslo ir veiksmo kodas atrodo taip:

```
class_name WanderGoal
extends Goal

# Override
func compute_reward(_agent: GdPAIAgent) -> float:
    return 20

# Override
func get_desired_state(agent: GdPAIAgent) -> Array[Precondition]:
```

```

    var agent_position: Vector2i = agent.blackboard.get_property("tilemap_position")

    var move_condition: Precondition = Precondition.new()
    move_condition.eval_func = func(blackboard: GdPAIBlackboard, _world_state: GdPAIBlackboard):
        var sim_position: Vector2i = blackboard.get_property("tilemap_position")

        var x_diff = abs(agent_position.x - sim_position.x)
        var y_diff = abs(agent_position.y - sim_position.y)

        return x_diff >= 1 || y_diff >= 1

    return [move_condition]

# Overload
func get_title() -> String:
    return "Wander around"

# Overload
func get_description() -> String:
    return "Move around the environment."

```

O klajojimo veiksmo kodas atrodo taip:

```

class_name WanderAction
extends Action

# Override
func get_validity_checks() -> Array[Precondition]:
    var checks: Array[Precondition] = []
    checks.append(Precondition.agent_has_property("entity"))
    checks.append(Precondition.agent_has_property("tilemap_position"))

    var is_not_surrounded: Precondition = Precondition.new()
    is_not_surrounded.eval_func = func(blackboard: GdPAIBlackboard, _world_state: GdPAIBlackboard):
        var agent_position: Vector2i = blackboard.get_property("entity").tilemap_position

        var available_neighbors = Globals.grid_system.get_neighbors(agent_position)

        return not available_neighbors.is_empty()
    checks.append(is_not_surrounded)

    return checks

# Override
func get_action_cost(
    _agent_blackboard: GdPAIBlackboard,
    _world_state: GdPAIBlackboard
) -> float:
    return 1

# Override
func get_preconditions() -> Array[Precondition]:
    return []

# Override
func simulate_effect(
    agent_blackboard: GdPAIBlackboard,
    _world_state: GdPAIBlackboard
):

```

```

    var sim_position: Vector2i = agent_blackboard.get_property("tilemap_position")
    sim_position.x += 1
    agent_blackboard.set_property("tilemap_position", sim_position)

# Override
func reverse_simulate_effect(
    _agent_blackboard: GdPAIBlackboard,
    _world_state: GdPAIBlackboard
):
    pass

# Override
func pre_perform_action(agent: GdPAIAgent) -> Action.Status:
    var agent_position: Vector2i = agent.blackboard.get_property("tilemap_position")

    var possible_targets = Globals.grid_system.get_navigable_neighbors(agent_position)
    if possible_targets.is_empty():
        SignalBus.action_done.emit()
        return Action.Status.FAILURE

    var target_location : Vector2i = possible_targets.pick_random()
    agent.blackboard.set_property(uid_property("target_position"), target_location)

    return Action.Status.SUCCESS

# Override
func perform_action(
    agent: GdPAIAgent,
    _delta: float
) -> Action.Status:
    if not agent.entity.is_active:
        return Action.Status.RUNNING

    var target_position: Vector2i = agent.blackboard.get_property(uid_property("target_position"))

    agent.entity.step(target_position)

    return Action.Status.SUCCESS

# Override
func post_perform_action(agent: GdPAIAgent) -> Action.Status:
    agent.blackboard.erase_property(uid_property("target_position"))
    return Action.Status.SUCCESS

# Override
func get_title() -> String:
    return "Wander"

```

Veiksma ir tikslo kodas nieko nežino apie vienas kitą, bet koks veiksmas kuris pajudintų agentą gali įvykdyti šį tikslą, skirtumas atsiranda tik veiksmų kainose.

Įvykdyti DI komponentai

Įvykdyta agentų navigacija / kelio radimas, bei veiksmų planavimas.

Projekto kodas yra GitHub repozitorijoje, atskiroje šakoje[5] nuo mano senesnio projekto, kurio bazę naudoju šiam.

5. Analizė ir refleksija

Galiausiai pavyko sukurti norimą sistemą. Agentai sėkmingai gali suplanuoti veiksmų sekas ir keisti savo planą pagal pasaulio pokyčius. Šią sistemą teisingai sukonfigūruoti yra tikrai sudėtingiau, nei kitas žaidimų DI sistemas (būsenų mašinos ar elgesio medžiai).

Padariau klaidą pasirinkdamas naudoti seną projektą kaip bazę šiam. Jame nenaudojau Godot navigacijos sistemos, todėl negalėjau pasinaudoti GdPlanningAI pateikiamu SpatialAction kodu. Tuo tarpu didžiausią dalį darbo laiko teko taisyti navigaciją, ji sukėlė daug problemų planavimo etape. Galiausiai teko judesį susieti su kiekvienu veiksmu, kitaip agentų planavimas neveikė. Viso to būčiau išvengęs, jei būčiau pradėjęs šviežią projektą ir jame naudojęs Godot navigacijos sistemą.

Tačiau džiaugiuosi, kad galiausiai pavyko pasiekti savo išsikeltą tikslą. Teko praeiti daug kodo iteracijų, reikėjo beveik visas sistemas pertvarkyti, to procese išmokau daugiau apie Godot variklio ir Gdscript kalbos niuansus. Kartu turiu žinias, kaip naudoti GdPlanningAI įskiepi savo ateities projektams.

Išvados

Nors ir galutinis rezultatas ne visiškai atitinka pradinio plano, visi esminiai elementai yra. Pavyko sukurti visus norimus pasaulio objektus, agentai sėkmingai sukuria planus, kaip sąveikauti su šiais objektais.

GOAP nėra lengvas metodas valdyti agentus - agentai veikia labai savarankiškai, neatsižvelgiant į kitų agentų veiklą. Tai gali sukelti įvairių problemų, pavyzdžiui su navigacija ar planų efektyvumu. Tas pats savarankiškumas leidžia kiekvienam agentui dinamiškai prisitaikyti prie situacijos, tai suteikia daugiau laisvės statant žaidimo pasaulį.

Literatūra ir šaltiniai

1. https://web.archive.org/web/20210810043252/https://www.gamasutra.com/blogs/TommyThompson/20200507/362417/Building_the_AI_of_FEAR_with_Goal_Oriented_Action_Planning.php#expand
2. https://dke.maastrichtuniversity.nl/m.winands/documents/CIG2016_HTN.pdf
3. <https://github.com/AidasLit/Hex-game-uni>
4. <https://github.com/WahahaYes/GdPlanningAI>
5. <https://github.com/AidasLit/Hex-game-uni/tree/goap>